

1. Create Database and Student Table

Question

Create a database CollegeDB and create a table Student(StudentID, Name, Age, Course). Insert at least 3 records and define primary key.

Complete Answer

```
CREATE DATABASE CollegeDB;
```

```
USE CollegeDB;
```

```
CREATE TABLE Student(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT,  
    Course VARCHAR(50)  
);
```

```
INSERT INTO Student VALUES  
(1, 'Tanmay', 19, 'CSE'),  
(2, 'Rahul', 20, 'AI'),  
(3, 'Sneha', 18, 'IT');
```

```
SELECT * FROM Student;
```

2. Employee Salary Greater Than 30000

Question

Create a table Employee(EmpID, Name, Department, Salary) and display employees whose salary is greater than 30000.

Complete Answer

```
CREATE TABLE Employee(  
    EmpID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Department VARCHAR(50),  
    Salary INT  
);
```

```
INSERT INTO Employee VALUES  
(1, 'Amit', 'HR', 25000),  
(2, 'Riya', 'IT', 45000),  
(3, 'Karan', 'Sales', 35000);
```

```
SELECT * FROM Employee  
WHERE Salary > 30000;
```

3. Sort Courses by Credits Descending

Question

Create a table Course(CourseID, CourseName, Credits) and display courses sorted by credits in descending order.

Complete Answer

```
CREATE TABLE Course(  
    CourseID INT PRIMARY KEY,
```

```
CourseName VARCHAR(50),  
Credits INT  
);
```

```
INSERT INTO Course VALUES  
(1, 'DBMS', 4),  
(2, 'Python', 3),  
(3, 'AI', 5);
```

```
SELECT * FROM Course  
ORDER BY Credits DESC;
```

4. Students Age Greater Than 20

Question

Create a Student table and display students whose age is greater than 20.

Complete Answer

```
CREATE TABLE Student(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

```
INSERT INTO Student VALUES  
(1, 'Tanmay', 19),  
(2, 'Riya', 21),  
(3, 'Aman', 22);
```

```
SELECT * FROM Student  
WHERE Age > 20;
```

5. Calculate Total Sales Amount

Question

Create Sales table and calculate total sales amount.

Complete Answer

```
CREATE TABLE Sales(  
    SaleID INT PRIMARY KEY,  
    ProductName VARCHAR(50),  
    Amount INT  
);  
  
INSERT INTO Sales VALUES  
(1, 'Laptop', 50000),  
(2, 'Mouse', 1000),  
(3, 'Keyboard', 2000);  
  
SELECT SUM(Amount) AS TotalSales  
FROM Sales;
```

6. Count Students in Each Course

Question

Create Student and CourseDetails tables and count students in each course.

Complete Answer

```
CREATE TABLE CourseDetails(  
    CourseID INT PRIMARY KEY,  
    CourseName VARCHAR(50)  
);  
  
CREATE TABLE Student(  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    CourseID INT,  
    FOREIGN KEY(CourseID) REFERENCES CourseDetails(CourseID)  
);  
  
INSERT INTO CourseDetails VALUES  
(1, 'CSE'),  
(2, 'AI');  
  
INSERT INTO Student VALUES  
(1, 'Tanmay', 1),  
(2, 'Rahul', 1),  
(3, 'Sneha', 2);  
  
SELECT CourseID, COUNT(*) AS TotalStudents  
FROM Student  
GROUP BY CourseID;
```

7. INNER JOIN

Question

Retrieve customer names with order details using INNER JOIN.

Complete Answer

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);  
  
CREATE TABLE Orders(  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    Amount INT,  
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)  
);  
  
INSERT INTO Customer VALUES  
(1, 'Tanmay'),  
(2, 'Rahul');  
  
INSERT INTO Orders VALUES  
(101, 1, 5000),  
(102, 2, 7000);  
  
SELECT Customer.Name, Orders.OrderID, Orders.Amount  
FROM Customer  
INNER JOIN Orders  
ON Customer.CustomerID = Orders.CustomerID;
```

8. LEFT JOIN

Question

Retrieve all customers including those without orders using LEFT JOIN.

Complete Answer

```
SELECT Customer.Name, Orders.OrderID, Orders.Amount
FROM Customer
LEFT JOIN Orders
ON Customer.CustomerID = Orders.CustomerID;
```

9. View Creation

Question

Create a view named Student_View.

Complete Answer

```
CREATE VIEW Student_View AS
SELECT * FROM Student;

SELECT * FROM Student_View;
```

10. Trigger

Question

Create a trigger that records salary updates in Employee_Audit table.

Complete Answer

```
CREATE TABLE Employee(
    EmpID INT PRIMARY KEY,
```

```

        Name VARCHAR(50),
        Salary INT
    );

CREATE TABLE Employee_Audit(
    EmpID INT,
    OldSalary INT,
    NewSalary INT
);

DELIMITER //

CREATE TRIGGER Salary_Update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Employee_Audit
    VALUES(OLD.EmpID, OLD.Salary, NEW.Salary);
END//

DELIMITER ;

INSERT INTO Employee VALUES
(1, 'Tanmay', 30000);

UPDATE Employee
SET Salary = 40000
WHERE EmpID = 1;

SELECT * FROM Employee_Audit;

```

12. Count Total Orders

Question

Create Orders table and count total orders.

Complete Answer

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(50)  
);  
  
CREATE TABLE Orders(  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    Amount INT,  
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)  
);  
  
INSERT INTO Customer VALUES  
(1,'Tanmay');  
  
INSERT INTO Orders VALUES  
(101,1,5000),  
(102,1,6000);  
  
SELECT COUNT(*) AS TotalOrders  
FROM Orders;
```

13. Pass and Fail Categories

Question

Display number of students in pass and fail categories using GROUP BY.

Complete Answer

```
CREATE TABLE Student(  
    RollNo INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Marks INT  
);  
  
INSERT INTO Student VALUES  
(1, 'Tanmay', 80),  
(2, 'Rahul', 30),  
(3, 'Sneha', 75);  
  
SELECT  
CASE  
WHEN Marks >= 40 THEN 'Pass'  
ELSE 'Fail'  
END AS Result,  
COUNT(*) AS TotalStudents  
FROM Student  
GROUP BY Result;
```

16. Procedure Even or Odd

Question

Write procedure to check even or odd number.

Complete Answer

```
DELIMITER //
```



```
CREATE PROCEDURE EvenOdd(IN num INT)  
BEGIN  
  
    IF num % 2 = 0 THEN
```

```
        SELECT 'Even Number';
    ELSE
        SELECT 'Odd Number';
    END IF;
```

```
END//
```

```
DELIMITER ;
```

```
CALL EvenOdd(10);
```

17. Function Greater Number

Question

Write function to find greater of two numbers.

Complete Answer

```
DELIMITER //
```

```
CREATE FUNCTION GreaterNum(a INT,b INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    IF a>b THEN
```

```
        RETURN a;
```

```
    ELSE
```

```
        RETURN b;
```

```
    END IF;
```

```
END//
```

```
DELIMITER ;
```

```
SELECT GreaterNum(10,20);
```

18. Function Square Number

Question

Write function to calculate square of a number.

Complete Answer

```
DELIMITER //
```

```
CREATE FUNCTION SquareNum(n INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
    RETURN n*n;
```

```
END//
```

```
DELIMITER ;
```

```
SELECT SquareNum(5);
```

19. Print Numbers 1 to 10

Question

Write PL/SQL block to print numbers from 1 to 10.

Complete Answer

```
BEGIN

    FOR i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;

END;
/
```

20. Display Even Numbers 1 to 20

Question

Write procedure to display even numbers between 1 to 20.

Complete Answer

```
DELIMITER //

CREATE PROCEDURE EvenNumbers()
BEGIN

    DECLARE i INT DEFAULT 1;

    WHILE i <= 20 DO

        IF i % 2 = 0 THEN
            SELECT i;
```

```
        END IF;

        SET i = i + 1;

    END WHILE;

END//

DELIMITER ;

CALL EvenNumbers();
```

21. Display Odd Numbers Using Continue

Question

Write procedure to display odd numbers between 1 to 20 using continue.

Complete Answer

```
DELIMITER //

CREATE PROCEDURE OddNumbers()
BEGIN

    DECLARE i INT DEFAULT 0;

    myloop: LOOP

        SET i = i + 1;

        IF i > 20 THEN
            LEAVE myloop;
        END IF;
```

```
        IF i % 2 = 0 THEN
            ITERATE myloop;
        END IF;

        SELECT i;

    END LOOP;

END//

DELIMITER ;

CALL OddNumbers();
```

22. Maximum and Minimum Marks

Question

Create Student table and find maximum and minimum marks.

Complete Answer

```
CREATE TABLE Student(
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

INSERT INTO Student VALUES
(1, 'Tanmay', 85),
(2, 'Rahul', 60),
(3, 'Sneha', 95);
```

```
SELECT  
MAX(Marks) AS MaximumMarks,  
MIN(Marks) AS MinimumMarks  
FROM Student;
```